Unit 2: Syntax Analysis (Parsing)

In **Unit 1**, the focus was on breaking the source code into individual units called **tokens** using lexical analysis. However, simply identifying tokens is not enough to understand a program. The compiler must also verify whether these tokens are arranged in a **meaningful and grammatically correct structure**. This responsibility is handled by the **Syntax Analysis phase**, also known as **Parsing**.

Syntax analysis takes the **token stream produced by the lexical analyzer** and checks whether it follows the grammatical rules of the programming language. If the structure is valid, the parser builds a **parse tree (or syntax tree)** that represents the hierarchical structure of the program. If the structure is invalid, it reports **syntax errors** such as missing semicolons, unmatched parentheses, or incorrect ordering of operators.

1. Context-Free Grammar (CFG)

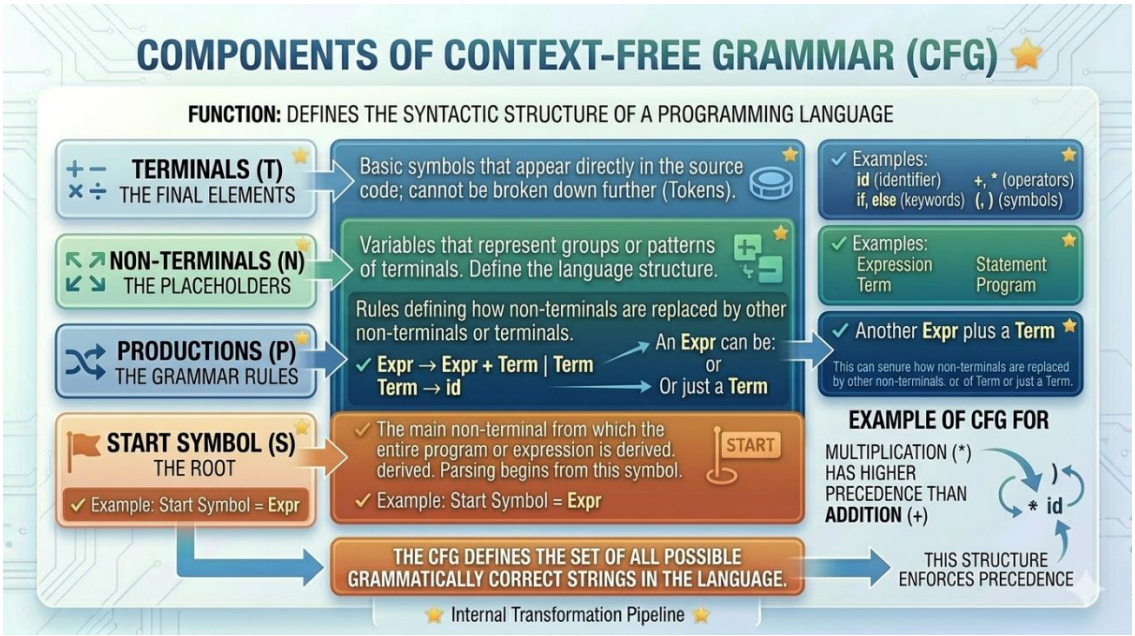
To define the syntax of programming languages, compilers use a formal system called a **Context-Free Grammar (CFG)**. Unlike regular expressions, which are suitable for simple patterns, CFGs are powerful enough to describe **nested and recursive structures**, such as:

- Balanced parentheses → (( ))
- Arithmetic expressions → a + b * c
- Nested control statements → if (x) { if (y) { ... } }

A CFG provides a set of rules that specify how valid sentences (programs) in a language can be formed.

Components of Context-Free Grammar

A Context-Free Grammar consists of **four main components**:



1. Terminals

Terminals are the **basic symbols** of the grammar. They represent the actual tokens that appear in the source code and cannot be broken down further.

✓ **Examples:**

- id (identifier)
- +, * (operators)
- if, else (keywords)
- (,) (symbols)

These are the **final elements** of the language.

2. Non-Terminals

Non-terminals are variables that represent groups or patterns of terminals. They are used to define the structure of the language and can be expanded into terminals or other non-terminals.

✓ **Examples:**

- Expression
- Term
- Statement
- Program

They act as **placeholders for syntactic structures**.

3. Productions (Grammar Rules)

Productions define how non-terminals can be replaced with other non-terminals or terminals. These are the rules that generate valid strings in the language.

✓ **Example:**

Expr → Expr + Term
Expr → Term
Term → id

✓ **Explanation:**

- An Expr can be:
  - Another Expr plus a Term
  - Or just a Term
- A Term can be an identifier (id)

These rules define how expressions are built.

4. Start Symbol

The **start symbol** is the main non-terminal from which the entire program is derived. It represents the complete structure of the language.

✓ **Example:**

Start Symbol = Expr
The parsing process begins from this symbol and expands using production rules.

Example of CFG for Arithmetic Expression

Let's take a simple example to understand CFG clearly:
Expr → Expr + Term | Term
Term → Term * Factor | Factor
Factor → (Expr) | id

✓ **Explanation:**

- Expr handles addition
- Term handles multiplication
- Factor handles identifiers and parentheses

This grammar ensures:

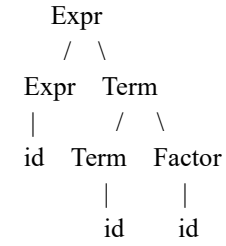
- Correct operator precedence (* before +)
- Proper handling of parentheses

How CFG Helps in Parsing

When the parser receives tokens like:

id + id * id

It uses CFG rules to check whether this sequence is valid and builds a structure like:



This structure is called a **parse tree**, which shows how the expression is derived from the grammar.

Importance of CFG

Context-Free Grammars are essential because they:

- ✓ Define the **syntax rules** of programming languages
- ✓ Help in building **parsers**
- ✓ Enable detection of **syntax errors**
- ✓ Support **nested and recursive structures**
- ✓ Form the foundation for **compiler design**

Detailed Example

Grammar:

Expr → Expr + Term | Term

Term → id

Components:

- **Terminals**: id, +
- **Non-terminals**: Expr, Term
- **Productions**: rules above
- **Start Symbol**: Expr

✓ Derivation Example:

Generate:

a + b + c

Expr

→ Expr + Term

→ Expr + Term + Term

→ Term + Term + Term

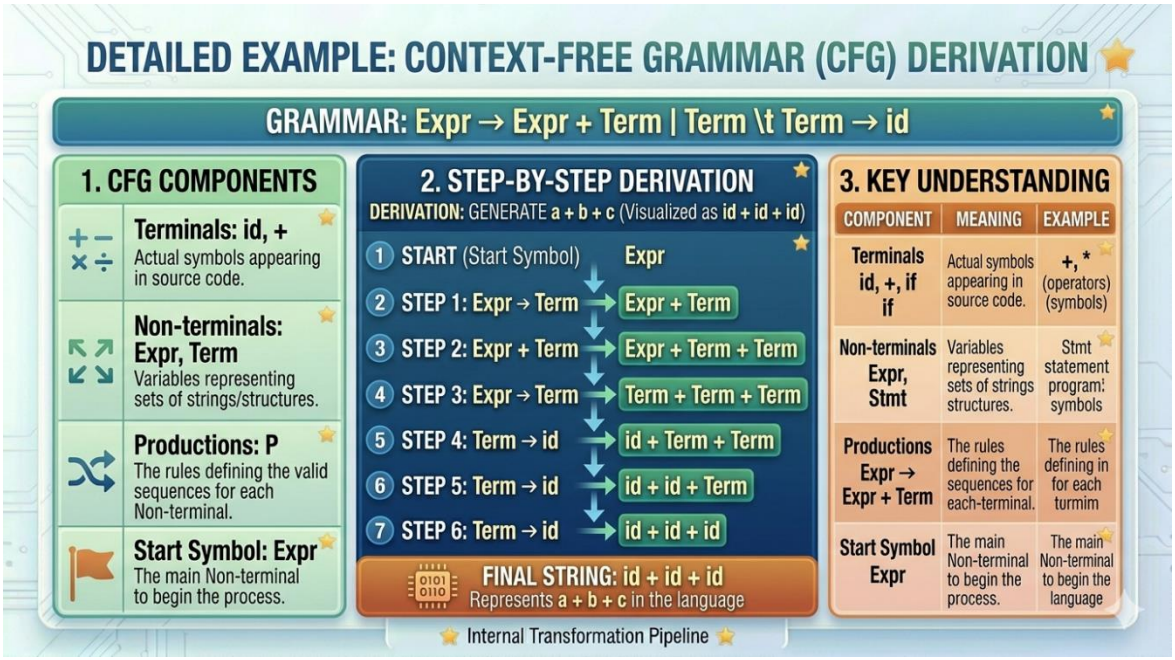
→ id + id + id

Final string:

id + id + id

Key Understanding

Component	Meaning	Example
Terminals	Actual symbols	id, +, if
Non-terminals	Variables representing structure	Expr, Stmt
Productions	Rules for replacement	Expr → Expr + Term
Start Symbol	Starting point of grammar	Expr



Parse Tree and Ambiguity

In the **syntax analysis phase**, once the compiler verifies that a sequence of tokens follows the grammar rules, it represents this structure using a **Parse Tree** (also called a **Syntax Tree**). This tree provides a **hierarchical and visual representation** of how a string (program) is derived from the grammar's start symbol. Along with this, an important concept called **ambiguity** arises when a grammar allows more than one interpretation of the same input. Let's understand both concepts in detail.

1. Parse Tree (Syntax Tree)

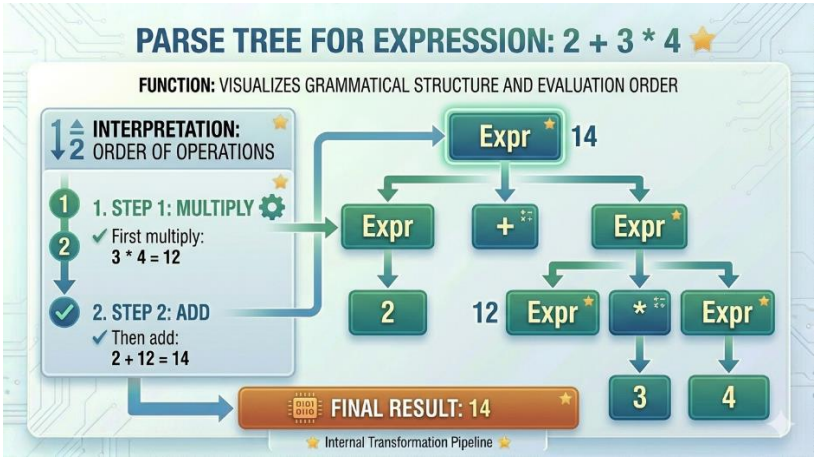
A **Parse Tree** is a tree-like structure that shows how a string is generated using the production rules of a **Context-Free Grammar (CFG)**. It starts from the **start symbol (root)** and expands using grammar rules until it reaches the **terminals (leaves)**.

✓ **Key Properties:**

- Root node → Start symbol
- Internal nodes → Non-terminals
- Leaf nodes → Terminals (actual tokens)
- Shows **step-by-step derivation** of a string

Example of Parse Tree
Consider the expression:
2 + 3 * 4
Using a simple grammar:
Expr → Expr + Expr
Expr → Expr * Expr
Expr → number

✓ Parse Tree Representation (One Possible Interpretation)



✓ Interpretation:

- First multiply: 3 * 4 = 12
- Then add: 2 + 12 = 14

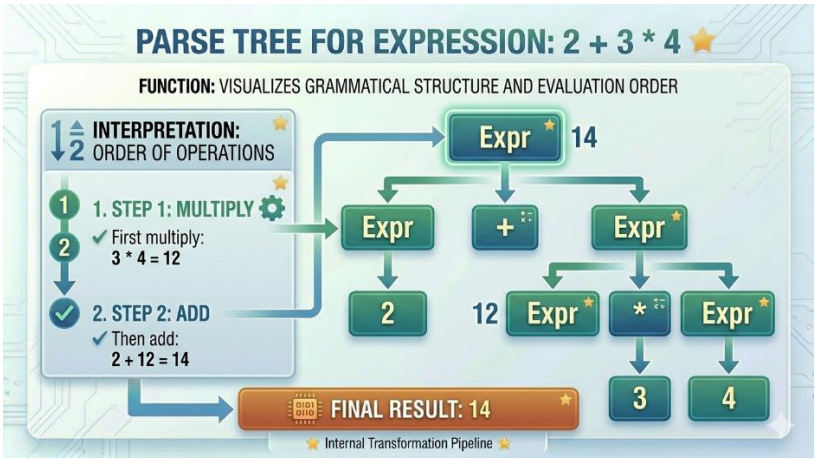
The parse tree clearly shows the **order of operations** and structure of evaluation.

2. Ambiguity in Grammar
A grammar is said to be **ambiguous** if a single input string can be represented by **more than one distinct parse tree**.
This means:

- The compiler has **multiple ways to interpret the same expression**
- Leads to **confusion in evaluation**

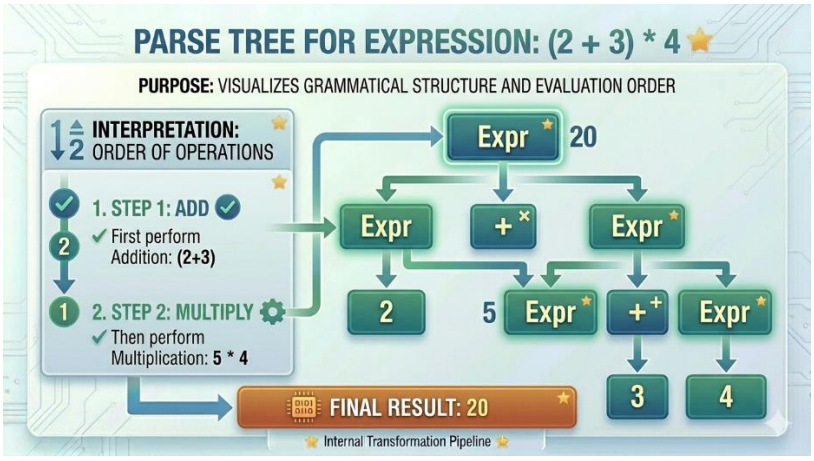
Example of Ambiguity
Consider the same expression:
2 + 3 * 4
Using the ambiguous grammar:
Expr → Expr + Expr
Expr → Expr * Expr
Expr → number

✓ **Parse Tree 1 (Multiplication First)**



Result:
2 + (3 * 4) = 14

✓ **Parse Tree 2 (Addition First)**



Result:
(2 + 3) * 4 = 20

Problem:
Same expression → **Two different results (14 and 20)**
This is **ambiguity**

3. Why Ambiguity is a Problem
Ambiguity is **undesirable in compilers** because:

- Compiler cannot decide correct evaluation
- Leads to inconsistent results
- Makes program behavior unpredictable
- Difficult to implement parsing algorithms

A programming language must have **unambiguous grammar**

4. Removing Ambiguity (Using Precedence)

To solve ambiguity, we define **operator precedence and associativity** directly in the grammar.

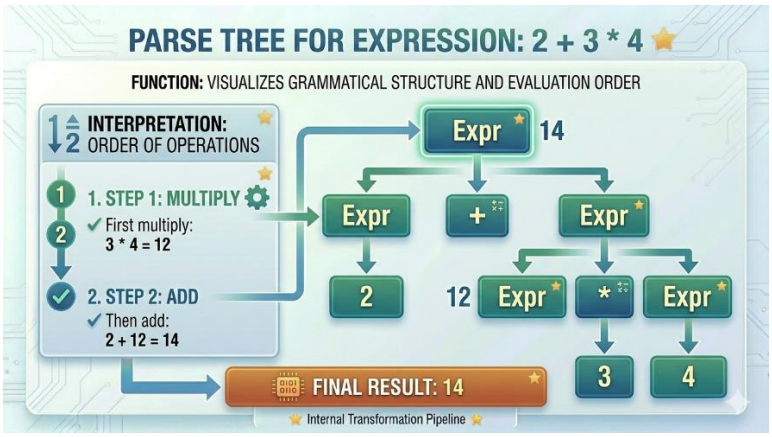
✓ Correct Grammar (Unambiguous)

Expr → Expr + Term | Term
Term → Term * Factor | Factor
Factor → number

✓ Explanation:

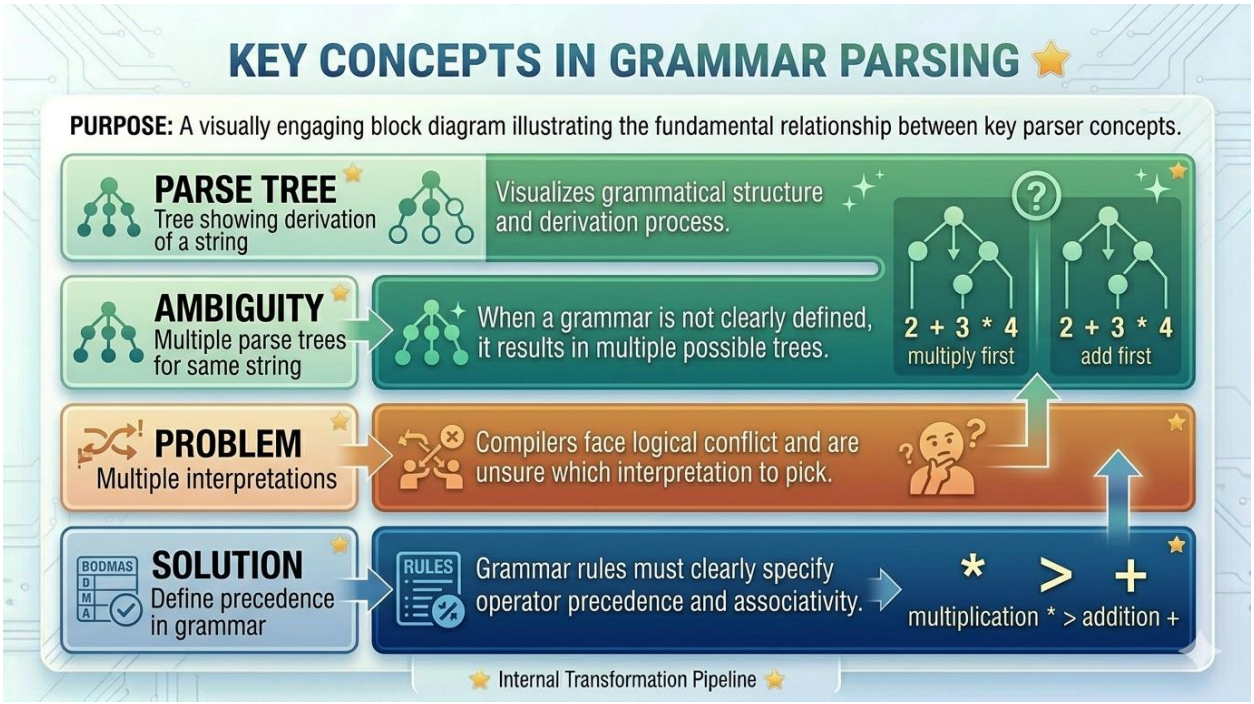
- * has higher precedence than +
- So multiplication is done first
- Grammar enforces correct structure

✓ Parse Tree (Unambiguous)



Correct result:
2 + (3 * 4) = 14

Key Concepts Summary



3. Top-Down Parsing:

Top-down parsing starts from the **root** (the start symbol) and tries to "build down" to reach the terminals (the tokens).

- **Recursive Descent:** This is a manual approach where each non-terminal in the grammar is written as a separate function. The functions call each other recursively. It's easy to write but can struggle with "Left Recursion" (where a rule starts with itself, causing an infinite loop).
- **LL(1) Parsing:** This is a more formal, table-driven approach.
    - **L:** Scans input from **L**eft to right.
    - **L:** Produces a **L**eftmost derivation.
    - **(1):** Uses **one** lookahead token to decide which rule to apply.
    -

Working Principle of Top-Down Parsing (Detailed Description with Parse Tree)

Top-down parsing is a fundamental technique used in syntax analysis where the parser begins with the **start symbol** of the grammar and attempts to derive the given input string step by step. The goal of the parser is to check whether the input string can be generated using the production rules of the grammar. In this approach, the parser constructs the parse tree **from the root (top) to the leaves (bottom)**, which is why it is called *top-down parsing*.

A key characteristic of top-down parsing is that it always performs a **leftmost derivation**. This means that at each step, the parser selects and expands the **leftmost non-terminal** in the current string. By repeatedly applying production rules, the parser tries to transform the start symbol into the input string. If it succeeds, the input is considered syntactically correct; otherwise, a syntax error is reported.

Example Grammar and Input

Consider the following grammar:

S → aA

A → b

Here:

- S is the **start symbol**
- a and b are **terminals**

- A is a **non-terminal**

✓ **Input String:**
ab

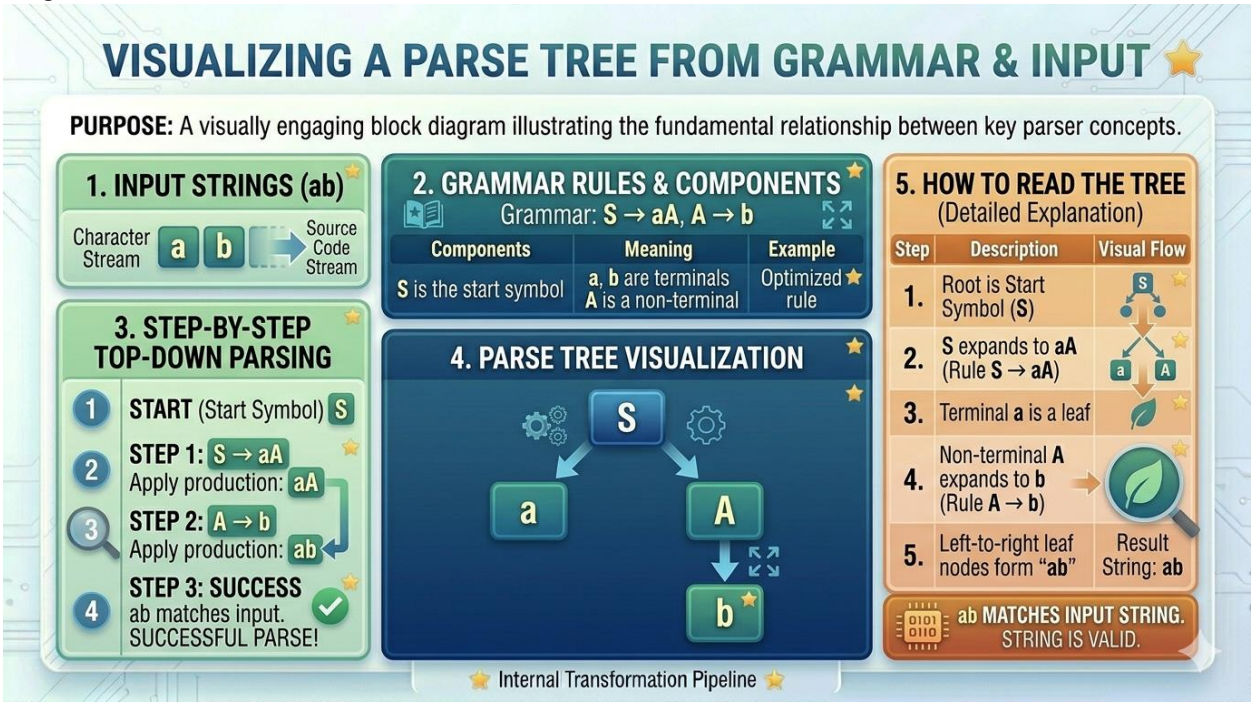
Step-by-Step Working of Top-Down Parsing
The parsing process begins with the start symbol S. The parser then applies production rules to gradually transform S into the input string.

✓ **Step 1:**
Start with:
S
Apply the production rule:
S → aA
Now the string becomes:
aA

✓ **Step 2:**
Next, expand the **leftmost non-terminal**, which is A:
A → b
Now the string becomes:
ab

✓ **Step 3:**
The derived string ab matches the input string.
Therefore, the parsing is **successful**.

Parse Tree Representation
The same derivation can be represented using a **parse tree**, which visually shows how the string is generated from the grammar.



Explanation of the Parse Tree

- The root node is the **start symbol (S)**
- S expands into aA using the rule S → aA
- The terminal a becomes a leaf node
- The non-terminal A is further expanded into b
- The leaf nodes (from left to right) form the string:

ab

This matches the input string, confirming that the string is valid.

Important Observations

- The parser always expands the **leftmost non-terminal first**
- The derivation proceeds from **top to bottom**
- The parse tree shows the **hierarchical structure** of the input
- If at any step no valid production rule can be applied, parsing fails

Intuitive Understanding

You can think of top-down parsing as **starting with a blueprint (start symbol)** and gradually refining it until it becomes the final product (input string). At each step, the parser makes decisions based on grammar rules to move closer to the target string.

Recursive Descent Parsing

Recursive Descent Parsing is one of the simplest and most intuitive techniques used in **top-down parsing**. In this approach, the parser is implemented as a set of **mutually recursive functions**, where each function corresponds to a **non-terminal** in the grammar. The parsing process begins with the start symbol and proceeds by calling these functions in a way that naturally constructs the **parse tree from top to bottom**.

The key idea behind recursive descent parsing is that the structure of the program being parsed is directly reflected in the structure of the function calls. As each function is invoked, it attempts to match a portion of the input string according to the grammar rules. If the match is successful, the function returns control to the calling function; otherwise, a syntax error is reported. This process continues until the entire input string is either successfully parsed or rejected.

Example Grammar

To understand this clearly, consider the following grammar (after removing left recursion):

E → T E'

E' → + T E' | ε

T → id

Here:

- E is the start symbol
- E' handles repetition of + operations
- T represents identifiers

Input String

id + id

Working of Recursive Descent Parsing

The parsing process begins by calling the function corresponding to the **start symbol**, i.e., E(). Each function tries to match a part of the input and may call other functions based on the grammar rules.

✓ **Function Call Flow**

E()
→ T()
→ match(id)
→ E'()
→ match(+)
→ T()
→ match(id)
→ E'()
→ ε

Step-by-Step Explanation

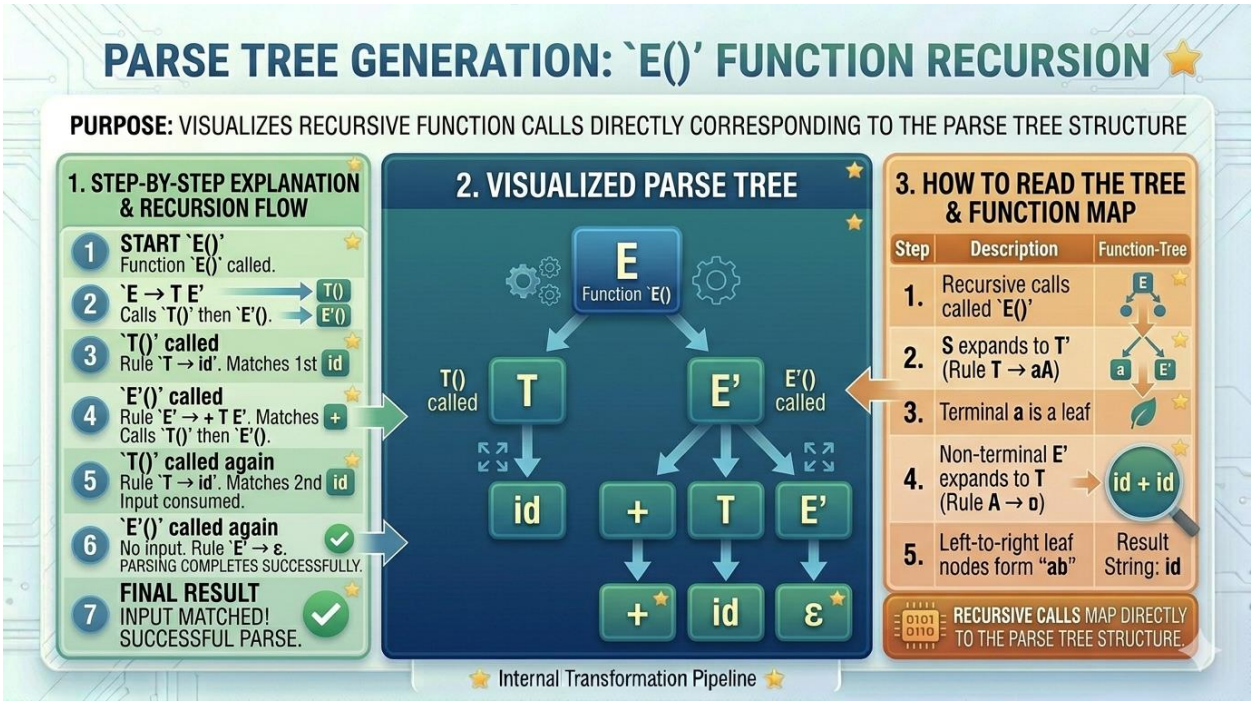
The process starts with the function E():

1. **E() is called**
   - According to the rule E → T E', it first calls T() and then E'()
2. **T() is called**
   - T → id, so it matches the first token id from the input
   - Input now becomes: + id
3. **E'() is called**
   - The next input symbol is +, so it follows the rule E' → + T E'
   - It matches + and calls T()
4. **T() is called again**
   - Matches the next id
   - Input is now fully consumed
5. **E'() is called again**
   - No more input remains, so it applies E' → ε (empty production)

Parsing completes successfully because the entire input string has been matched.

Parse Tree Representation

The recursive function calls directly correspond to the structure of the **parse tree**:



Understanding the Tree

- The root node is **E (start symbol)**
- E expands into T and E'
- T produces id
- E' expands into + T E'
- The second T produces another id
- The final E' becomes **ε (empty)**

The leaf nodes (left to right) form:

id + id

Key Insight: Function Calls = Tree Nodes

One of the most important observations is:

Each function call represents a node in the parse tree

- E() → Root node
- T() → Child node
- E'() → Subtree
- match(id) → Leaf node

As functions call each other recursively, they **build the tree structure implicitly**.

Another Example (Simple Case)

Grammar:

S → aA

A → b

Input:

ab

Function Calls:

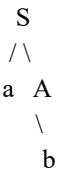
S()

→ match(a)

→ A()

→ match(b)

Parse Tree:



LL(1) Parsing

LL(1) Parsing is a structured and efficient **top-down parsing technique** used in compiler design. Unlike recursive descent (which uses functions), LL(1) parsing is a **table-driven method** that uses a parsing table to decide which production rule to apply at each step. It is also called a **predictive parser** because it can predict the correct rule using only limited information.

Meaning of LL(1)

The term **LL(1)** has a precise meaning:

- **First L (Left to Right)** → The input is scanned from **left to right**

- **Second L (Leftmost Derivation)** → The parser constructs a **leftmost derivation**
- **1 (One Lookahead)** → The parser uses **only one next input token** to make decisions

This makes LL(1) parsing **fast, deterministic, and efficient**.

Working Principle of LL(1) Parsing

LL(1) parsing uses three main components:

- **Input Buffer** → Contains the input string
- **Stack** → Used to keep track of grammar symbols
- **Parsing Table** → Guides which production rule to apply

The parsing process starts with the **start symbol on the stack** and tries to match the input string step by step.

At each step:

1. Look at the **top of the stack**
2. Look at the **current input symbol (lookahead)**
3. Use the parsing table to decide which rule to apply
4. Replace non-terminals or match terminals

Example Grammar (LL(1) Form)

E → T E'

E' → + T E' | ε

T → id

Input String

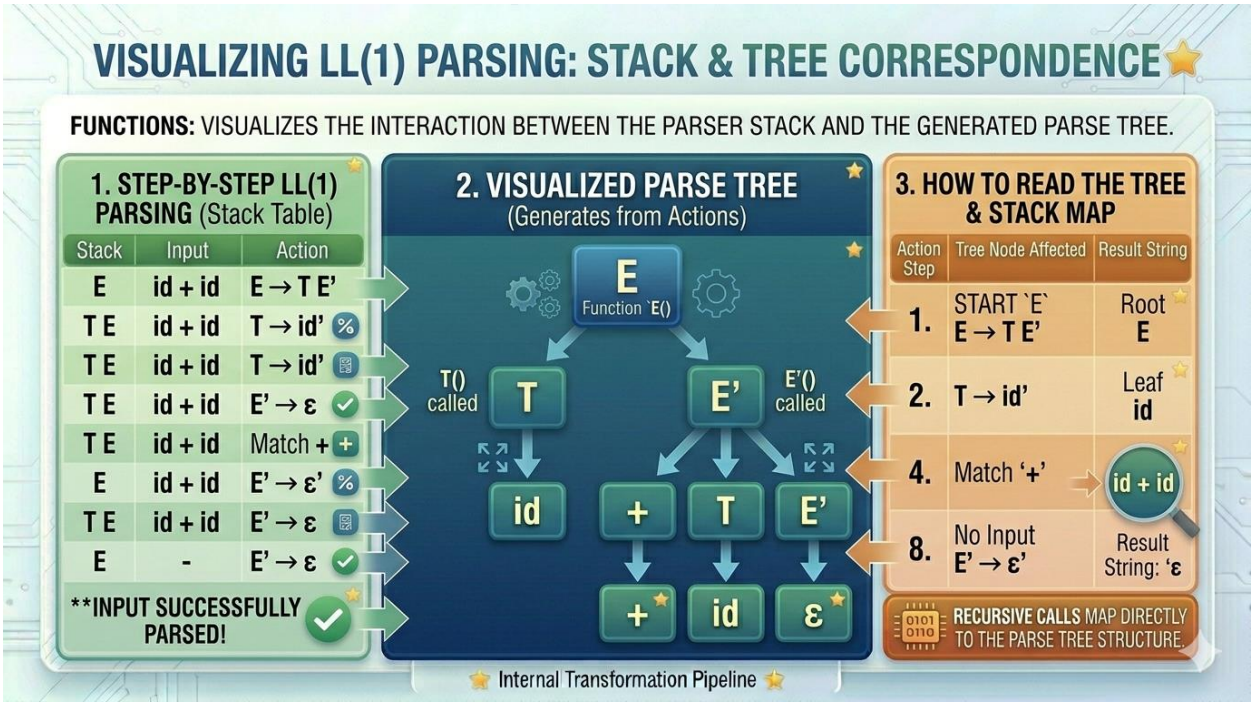
id + id

Step-by-Step LL(1) Parsing

Stack	Input	Action
E	id + id	E → T E'
T E'	id + id	T → id
id E'	id + id	Match id
E'	+ id	E' → + T E'
+ T E'	+ id	Match +
T E'	id	T → id
id E'	id	Match id
E'	—	E' → ε

Input is successfully parsed

Parse Tree Representation



Explanation of the Parse Tree

- Start symbol E expands into T E'
- T produces id
- E' expands into + T E'
- Second T produces id
- Final E' becomes **ε (empty)**

Leaf nodes form:
id + id

Why LL(1) is Efficient?

LL(1) parsing is efficient because:

✓ No backtracking is required

✓ Only one lookahead symbol is enough

✓ Uses precomputed parsing table

✓ Time complexity is linear (O(n))

Conditions for LL(1) Grammar

For a grammar to be LL(1), it must satisfy:

✓ **No Left Recursion**

✓ **Left Factoring required**

✓ No ambiguity

FIRST and FOLLOW Sets

In **LL(1) parsing**, the parser must decide **which production rule to apply** at each step using only **one lookahead token**. To make this decision correctly and without ambiguity, two important mathematical concepts are used: **FIRST sets** and **FOLLOW sets**. These sets provide essential information about the grammar and help in constructing the **LL(1) parsing table**.

1. FIRST Set

The **FIRST(A)** of a non-terminal **A** is the set of **terminals that can appear at the beginning** of any string derived from A.
FIRST tells us **what symbols can come first** when we expand a non-terminal.

Rules to Compute FIRST

1. If **A → aα** (where a is a terminal), then
   → a ∈ FIRST(A)
2. If **A → Bα**, then
   → Add all elements of FIRST(B) into FIRST(A)
3. If **A → ε (empty string)**, then
   → ε ∈ FIRST(A)

Example Grammar

E → T E'
E' → + T E' | ε
T → id

FIRST Set Calculation

✓ **FIRST(T)**
T → id
So,
FIRST(T) = { id }

FIRST(E')
E' → + T E' | ε
So,
FIRST(E') = { +, ε }

✓ **FIRST(E)**
E → T E'
Since FIRST(T) = { id }
FIRST(E) = { id }

2. FOLLOW Set

The **FOLLOW(A)** of a non-terminal **A** is the set of terminals that can appear **immediately after A** in some valid string.
FOLLOW tells us **what can come next after A**

Rules to Compute FOLLOW

1. Place $ (end marker) in FOLLOW(Start Symbol)
2. If **A → αBβ**, then
   → Add FIRST(β) (excluding ε) to FOLLOW(B)
3. If **A → αB** OR FIRST(β) contains ε, then
   → Add FOLLOW(A) to FOLLOW(B)

FOLLOW Set Calculation

✓ **Step 1: Start Symbol**
FOLLOW(E) = { $ }

✓ **FOLLOW(E')**
From:
E → T E'
So:
FOLLOW(E') = FOLLOW(E) = { $ }

✓ **FOLLOW(T)**
From:
E → T E'

- FIRST(E') = { +, ε }

So:
FOLLOW(T) = { + } ∪ FOLLOW(E)
= { +, $ }

Final FIRST and FOLLOW Sets

Non-Terminal	FIRST	FOLLOW
E	{ id }	{ $ }
E'	{ +, ε }	{ $ }
T	{ id }	{ +, $ }

Role in LL(1) Parsing Table
These sets are used to decide **which production to apply**:

- Use **FIRST** to decide based on input symbol
- Use **FOLLOW** when ε (empty) production is involved

Example Decision
For:
E' → ε
We apply this rule when:
lookahead ∈ FOLLOW(E') = { $ }

Why FIRST and FOLLOW are Important

- ✓ Help in **constructing LL(1) parsing table**
- ✓ Avoid ambiguity and backtracking
- ✓ Enable **predictive parsing**
- ✓ Ensure correct rule selection

Bottom-Up Parsing: Shift-Reduce and LR Basics

In contrast to top-down parsing, which starts from the **start symbol and expands downward**, **bottom-up parsing** works in the opposite direction. It begins with the **input tokens** and gradually combines them (reduces them) to reach the **start symbol**. This approach is often described as **"building the parse tree from leaves to root"**.

In simple terms:
Top-down = *Start → Input*
Bottom-up = *Input → Start*

Working Principle of Bottom-Up Parsing
Bottom-up parsing tries to **reverse the derivation process**. Instead of generating the input string, it attempts to **reduce the input string back to the start symbol** using grammar rules.
It follows a **Rightmost Derivation in Reverse**.

1. Shift-Reduce Parsing
The most common implementation of bottom-up parsing is **Shift-Reduce Parsing**. It uses a **stack** and an **input buffer** to process the string.

Two Main Operations
✓ **Shift**

- Move the next input symbol onto the stack

✓ **Reduce**

- If the top of the stack matches the **right-hand side (RHS)** of a production rule
- Replace it with the **left-hand side (LHS)**

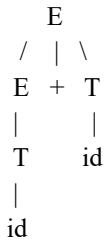
Example Grammar
E → E + T | T
T → id

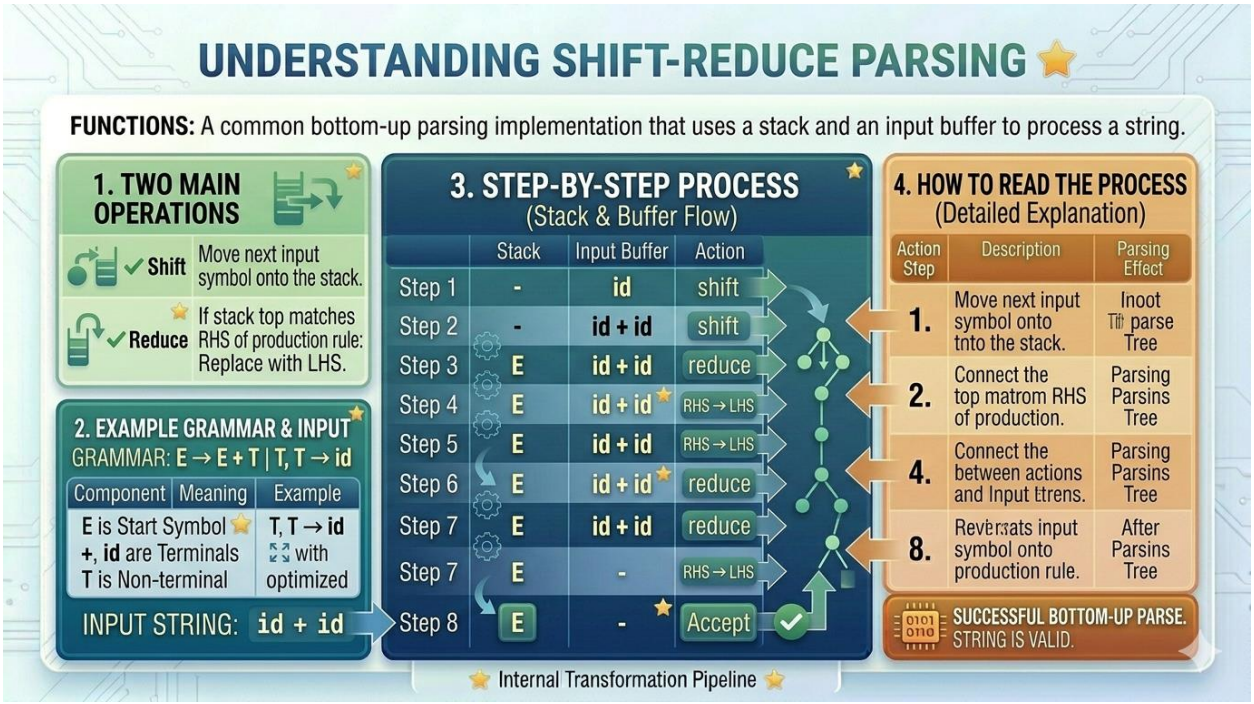
Input String
id + id

Step-by-Step Shift-Reduce Process

Stack	Input	Action
—	id + id	Shift
id	+ id	Reduce T → id
T	+ id	Reduce E → T
E	+ id	Shift
E +	id	Shift
E + id	—	Reduce T → id
E + T	—	Reduce E → E + T
E	—	Accept

Parse Tree (Bottom-Up Construction)





2. LR Parsing (Advanced Bottom-Up Parsing)

LR parsing is a more powerful and systematic version of shift-reduce parsing. It is also **table-driven**, similar to LL(1), but much more capable.

Meaning of LR

- **L (Left to Right)** → Input is scanned from left to right
- **R (Rightmost Derivation)** → Produces **rightmost derivation in reverse**

Key Idea

LR parsers:

- Use a **stack + parsing table**
- Decide whether to **shift or reduce**
- Handle **more complex grammars than LL(1)**

Types of LR Parsers

1. **SLR (Simple LR)**
2. **LALR (Look-Ahead LR)**
3. **Canonical LR (CLR)**

Increasing order of power:

SLR < LALR < Canonical LR

Why LR Parsers are Powerful

✓ Handle **left recursion directly**

✓ Work with **larger class of grammars**

✓ No need for grammar transformation (like LL(1))

✓ Detect errors more accurately

Shift-Reduce vs LL (1)

Feature	LL(1) (Top-Down)	LR (Bottom-Up)
Direction	Top → Down	Bottom → Up
Derivation	Leftmost	Rightmost (reverse)
Power	Less	More
Left Recursion	Not allowed	Allowed
Complexity	Simple	Complex

Intuitive Understanding

Think of bottom-up parsing like **assembling a puzzle**:

- You start with **small pieces (tokens)**
- Combine them step-by-step
- Finally form the **complete picture (start symbol)**

Key Points

- ✓ Bottom-up = **Reduce input to start symbol**
- ✓ Uses **Shift and Reduce operations**
- ✓ Builds **parse tree from leaves to root**
- ✓ LR parsers are **more powerful than LL(1)**

Explanation of Tree Construction

- Start with **leaf nodes (tokens)**: id + id
- Reduce id → T → E
- Reduce second id → T
- Combine using rule E → E + T
- Finally reach **start symbol (E)**

Tree is built **from bottom (tokens) to top (start symbol)**

Error Handling in Parsing

During syntax analysis, the parser checks whether the sequence of tokens follows the grammar rules of the programming language. However, in real-world programming, it is very common for programmers to make mistakes such as **missing semicolons, unmatched brackets, or incorrect syntax**. In such cases, the parser must not only detect the error but also handle it effectively. This process is known as **error handling in parsing**.

The primary goal of error handling is twofold:

First, the parser should **report the error clearly and accurately**, including details like the type of error and its location (line number). Second, it should **recover from the error and continue parsing the remaining input**, so that multiple errors can be identified in a single compilation rather than stopping at the first error.

Need for Error Handling

If the parser stops immediately after detecting an error, the programmer would have to fix errors one by one, which is inefficient. Therefore, modern compilers are designed to:

✓ Detect errors early

✓ Provide meaningful error messages

✓ Continue parsing to find more errors

This improves **developer productivity and debugging efficiency**

Types of Syntax Errors

Some common syntax errors include:

- Missing semicolon → int x = 10
- Unmatched brackets → (a + b
- Incorrect operator usage → a + * b
- Missing keywords → if x > 0

Error Handling Strategies

To handle errors effectively, parsers use different recovery techniques. The most common strategies are:

1. Panic Mode Recovery

Panic mode is the **simplest and most widely used error recovery technique**.

✓ **Working:**

- When an error is detected, the parser **discards input tokens** one by one
- It continues discarding until it finds a **synchronizing token**
- Synchronizing tokens are symbols like:
    - ; (end of statement)
    - } (end of block)

Example:

int x = 10
int y = 20;

Error:

Missing semicolon after 10

Panic Mode Action:

- Parser detects error
- Skips tokens until it finds ;
- Resumes parsing from:

int y = 20;

Advantages:

- Simple to implement
- Guarantees recovery

Disadvantages:

- May skip important parts of code
- Error messages may be less precise

2. Phrase-Level Recovery

Phrase-level recovery is a **more intelligent approach** where the parser tries to **fix the error locally** instead of skipping large portions of input.

✓ **Working:**

- Parser makes **small corrections** such as:
    - Inserting missing tokens
    - Deleting extra tokens
    - Replacing incorrect tokens

✓ **Example 1 (Missing Semicolon):**
int x = 10
Parser may assume:
int x = 10;

✓ **Example 2 (Missing Parenthesis):**
if (x > 0
Parser may correct it to:
if (x > 0)

Advantages:

- Produces **better error messages**
- Minimizes loss of correct input

Disadvantages:

- More complex to implement
- May sometimes guess incorrectly

Comparison of Strategies

Feature	**Panic Mode**	**Phrase-Level Recovery**
Approach	Skip tokens	Fix locally
Complexity	Simple	Complex
Accuracy	Low	Higher
Code Loss	More	Less

Key Responsibilities of Parser

When an error occurs, the parser must:

- ✓ Identify the error
- ✓ Report it clearly
- ✓ Recover from it
- ✓ Continue parsing